

Universidade Federal de Santa Catarina · Centro Tecnológico
Departamento de Informática e Estatística · Ciência da Computação

Polyglot Import CSV

Uma solução para importação de documentos CSV para múltiplos bancos de dados

Lucas Bueno Cesario
Orientador: Prof. Dr. Ronaldo dos Santos Mello

Roteiro

1. Contexto e trabalhos relacionados
2. A ferramenta Polyglot Import CSV
3. Avaliação de desempenho
4. Conclusão

Roteiro

1 **Contexto e trabalhos relacionados**

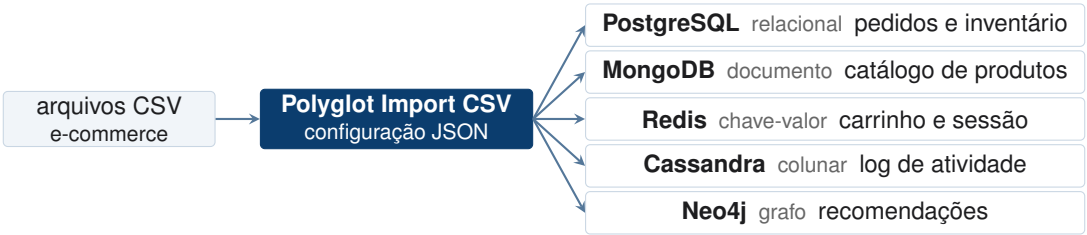
2 A ferramenta Polyglot Import CSV

3 Avaliação de desempenho

4 Conclusão

Persistência poliglota e o problema

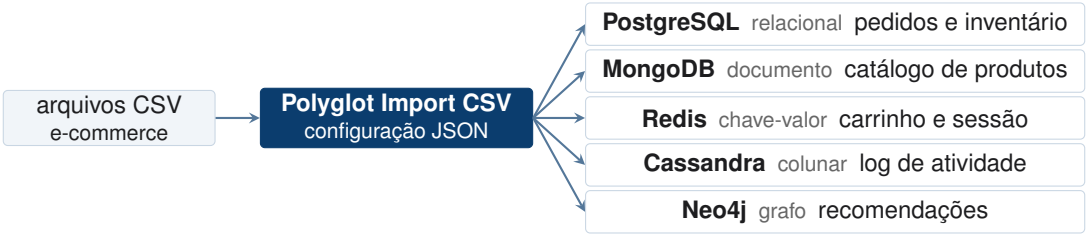
- **Persistência poliglota**: cada tipo de dado fica no SGBD (sistema gerenciador de banco de dados) mais adequado.
- Cada SGBD tem o seu **importador nativo** de CSV, que atende só a ele.



Cada dado do e-commerce no SGBD mais adequado (Figura 1 do relatório).

Persistência poliglota e o problema

- **Persistência poliglota**: cada tipo de dado fica no SGBD (sistema gerenciador de banco de dados) mais adequado.
- Cada SGBD tem o seu **importador nativo** de CSV, que atende só a ele.
- **Problema:** nenhuma ferramenta encontrada importa o mesmo CSV nos cinco.



Cada dado do e-commerce no SGBD mais adequado (Figura 1 do relatório).

Objetivos

Objetivo geral

Desenvolver a ferramenta *Polyglot Import CSV*, que importa CSV para múltiplos SGBDs: PostgreSQL, Redis, MongoDB, Cassandra ou Neo4j.

Objetivos específicos

1. **Configuração** em JSON (texto estruturado): filtros, chave primária e nome da coluna.
2. **Validação prévia**: um erro na configuração ou nos dados impede qualquer gravação.
3. **Avaliação** do desempenho com diferentes volumes e modelos de dados.
4. **Acesso** por linha de comando (CLI) e por interface gráfica (GUI).

Trabalhos relacionados

Comparação com os trabalhos relacionados

✓ o Cap. 3 afirma — o Cap. 3 não afirma

Trabalho	Foco	Destinos	Entrada CSV	Mapeamento declarativo validado	Revisão sem gravar
Importadores nativos	importação	um SGBD por vez	✓[1]	—	—
ArangoDB e `arangoimport`	importação	um SGBD multimodelo	✓	—	—
dbcrossbar	migração	PostgreSQL, BigQuery, nuvem	✓	—	—
Hackolade	modelagem visual	dezenas de alvos	—	—	—
Frameworks de acesso[2]	acesso em tempo de execução	vários SGBDs	—	—	—
Polystores[3] (BigDAWG)	consulta federada	fontes heterogêneas	—	—	—

[1] Redis: arquivo em protocolo bruto.
[2] Interface para ler e gravar em vários SGBDs (PEREIRA; GUERRA; ROSA, 2025; SACHDEVA; VASAVA, 2024).
[3] Uma só consulta sobre vários SGBDs (consulta federada).

Trabalhos relacionados

Comparação com os trabalhos relacionados ✓ o Cap. 3 afirma — o Cap. 3 não afirma

Trabalho	Foco	Destinos	Entrada CSV	Mapeamento declarativo validado	Revisão sem gravar
Importadores nativos	importação	um SGBD por vez	✓[1]	—	—
ArangoDB e `arangoimport`	importação	um SGBD multimodelo	✓	—	—
dbcrossbar	migração	PostgreSQL, BigQuery, nuvem	✓	—	—
Hackolade	modelagem visual	dezenas de alvos	—	—	—
Frameworks de acesso[2]	acesso em tempo de execução	vários SGBDs	—	—	—
Polystores[3] (BigDAWG)	consulta federada	fontes heterogêneas	—	—	—
Polyglot Import CSV	**importação**	**cinco SGBDs, cinco paradigmas**	✓	✓ **(JSON Schema)**	✓ **(`--dry-run`)**

[1] Redis: arquivo em protocolo bruto.
[2] Interface para ler e gravar em vários SGBDs (PEREIRA; GUERRA; ROSA, 2025; SACHDEVA; VASAVA, 2024).
[3] Uma só consulta sobre vários SGBDs (consulta federada).

Roteiro

1 Contexto e trabalhos relacionados

2 A ferramenta Polyglot Import CSV

3 Avaliação de desempenho

4 Conclusão

Visão geral

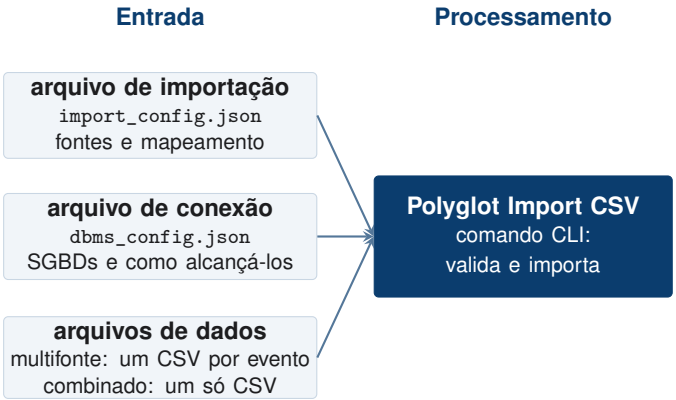
Entrada

arquivo de importação
`import_config.json`
fontes e mapeamento

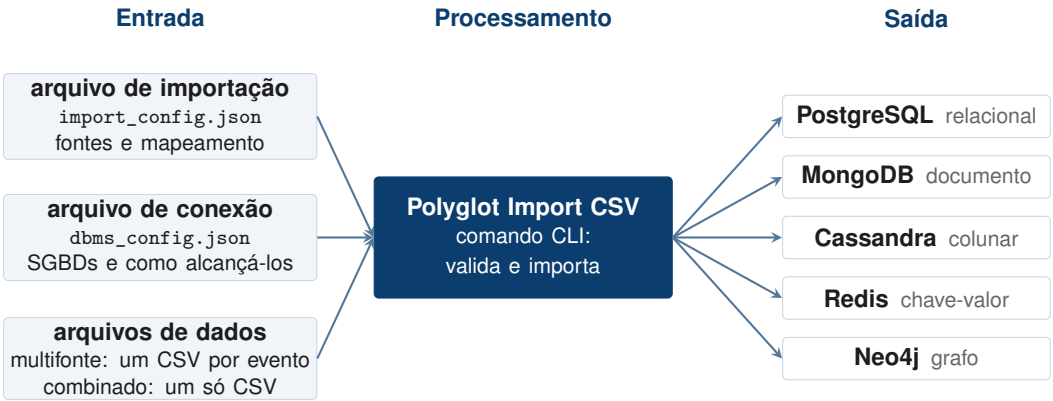
arquivo de conexão
`dbms_config.json`
SGBDs e como alcançá-los

arquivos de dados
multifonte: um CSV por evento
combinado: um só CSV

Visão geral


Entrada
Processamento
arquivo de importação
import_config.json
fontes e mapeamento
arquivo de conexão
dbms_config.json
SGBDs e como alcançá-los
arquivos de dados
multifonte: um CSV por evento
combinado: um só CSV
Polyglot Import CSV
comando CLI:
valida e importa

Visão geral



O comando valida as configurações e os dados e só então grava nos SGBDs destino.

Configuração em dois arquivos JSON

Importação: as fontes (CSVs) e o que gravar em cada SGBD. **Conexão**: onde alcançá-los.

Trecho do arquivo de importação: a fonte `stock` alimenta a tabela `inventory`.

```
"sources": {
  "stock": "ecommerce_stock.csv",
  "purchase": "ecommerce_purchase.csv", ...
},
"postgres": {
  "entities": { ...
    "inventory": {
      "source": "stock",
      "columns": {
        "product_id": { "is_key": true, "db_type": "BIGINT" },
        "quantity_available": { "db_type": "BIGINT" }, ...
      } }, ...
  }, ...
}, ...
```

- `sources`: dá um nome a cada CSV.
- `postgres`: um bloco por SGBD destino.
- `entities`: tabelas, coleções, nós...
- `columns`: colunas, com chave (`is_key`) e tipo (`db_type`).

Validação prévia em três etapas

A validação prévia tem **três etapas**, todas antes de gravar:

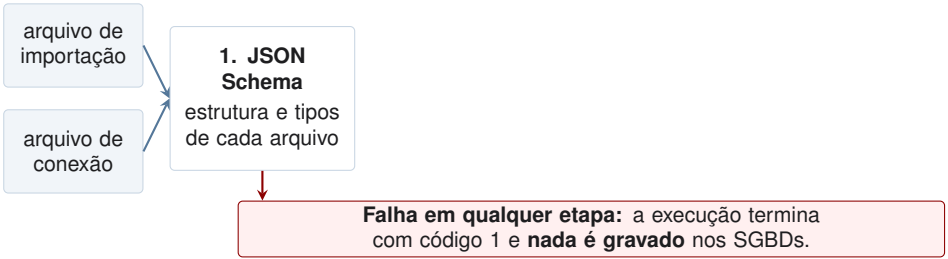
arquivo de importação

arquivo de conexão

Etapas da validação prévia, dos arquivos de configuração à gravação (Figura 10 do relatório).

Validação prévia em três etapas

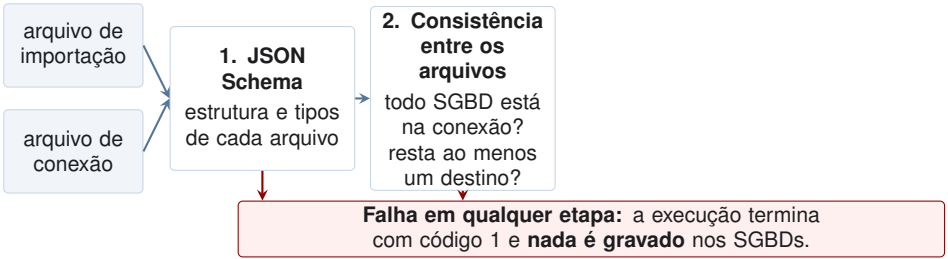
A validação prévia tem **três etapas**, todas antes de gravar:



Etapas da validação prévia, dos arquivos de configuração à gravação (Figura 10 do relatório).

Validação prévia em três etapas

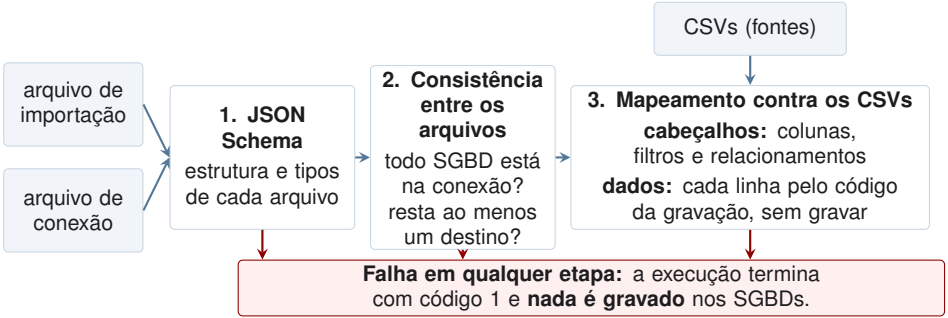
A validação prévia tem **três etapas**, todas antes de gravar:



Etapas da validação prévia, dos arquivos de configuração à gravação (Figura 10 do relatório).

Validação prévia em três etapas

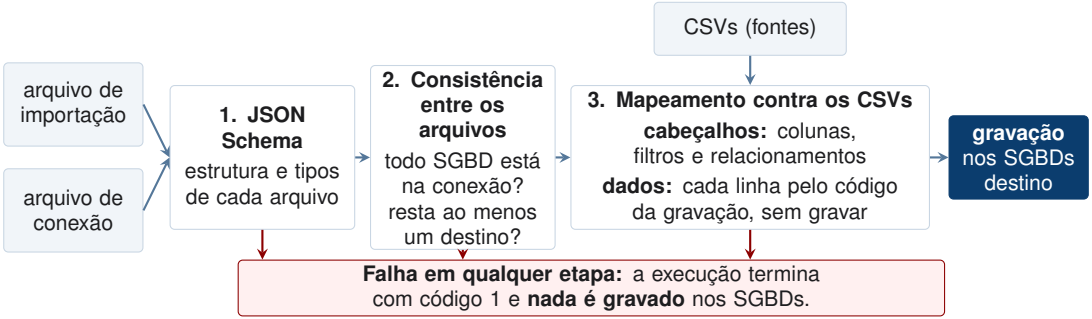
A validação prévia tem **três etapas**, todas antes de gravar:



Etapas da validação prévia, dos arquivos de configuração à gravação (Figura 10 do relatório).

Validação prévia em três etapas

A validação prévia tem **três etapas**, todas antes de gravar:



Etapas da validação prévia, dos arquivos de configuração à gravação (Figura 10 do relatório).

O mesmo cenário em cinco modelos de dados

Fio condutor: `user1` (`user_id` `7fd7...`) e `Product2` (`product_id` 2) ← indica a fonte (CSV)

PostgreSQL
relacional

← stock
inventory

product_id	2
quantity_available	2
last_restock_date	02/11/2023 03:30
price	50.50

O mesmo cenário em cinco modelos de dados

Fio condutor: `user1` (`user_id` `7fd7...`) e `Product2` (`product_id` 2) ← indica a fonte (CSV)

PostgreSQL
relacional

← stock
inventory

product_id	2
quantity_available	2
last_restock_date	02/11/2023 03:30
price	50.50

MongoDB
documento

← stock
product_catalog

```
{"product_id": 2,
 ...,
 "category": {...},
 "stock": {
  "quantity_available":
   2, ...}}
```

O mesmo cenário em cinco modelos de dados

Fio condutor: `user1` (`user_id 7fd7...`) e `Product2` (`product_id 2`) ← indica a fonte (CSV)

PostgreSQL
relacional

← stock
inventory
product_id 2
quantity_available 2
last_restock_date
02/11/2023 03:30
price 50.50

MongoDB
documento

← stock
product_catalog
{"product_id": 2,
...,
"category": {...},
"stock": {
"quantity_available":
2, ...}}

Cassandra
colunar

← as quatro fontes
user_activity_log
partição do user1
user_id = 7fd7...
event_time
event_type
02/11
03:30 stock
15/11
15:09 select_product
15:17 add_to_cart
15:46 purchase

O mesmo cenário em cinco modelos de dados

Fio condutor: `user1` (`user_id` `7fd7...`) e `Product2` (`product_id` 2) ← indica a fonte (CSV)

PostgreSQL
relacional

← stock
inventory
product_id 2
quantity_available 2
last_restock_date
02/11/2023 03:30
price 50.50

MongoDB
documento

← stock
product_catalog
{"product_id": 2,
...,
"category": {...},
"stock": {
"quantity_available":
2, ...}}

Cassandra
colunar

← as quatro fontes
user_activity_log
partição do user1
user_id = 7fd7...
event_time event_type
02/11
03:30 stock
15/11
15:09 select_product
15:17 add_to_cart
15:46 purchase

Redis
chave-valor

← add_to_cart
shopping_cart
chave user:7fd7...:cart
valor {...,
"cart_product_id": 3,
"cart_quantity": 1}

← select_product
user_session
chave 7fd7...
valor {"user_name":
"user1", ...}

O mesmo cenário em cinco modelos de dados

Fio condutor: `user1` (`user_id` `7fd7...`) e `Product2` (`product_id` 2) ← indica a fonte (CSV)

PostgreSQL
relacional

← stock
inventory
product_id 2
quantity_available 2
last_restock_date
02/11/2023 03:30
price 50.50

MongoDB
documento

← stock
product_catalog
{"product_id": 2,
...,
"category": {...},
"stock": {
"quantity_available":
2, ...}}

Cassandra
colunar

← as quatro fontes
user_activity_log
partição do user1
user_id = 7fd7...
event_time event_type
02/11
03:30 stock
15/11
15:09 select_product
15:17 add_to_cart
15:46 purchase

Redis
chave-valor

← add_to_cart
shopping_cart
chave user:7fd7...:cart
valor {...,
"cart_product_id": 3,
"cart_quantity": 1}

← select_product
user_session
chave 7fd7...
valor {"user_name":
"user1", ...}

Neo4j
grafo

← stock e purchase
(:User {...,
user_name: "user1",
...})
-[:PURCHASED {
order_number:
"ORD1", ...}]->
(:Product {
product_id: 2, ...})

Fluxo de execução

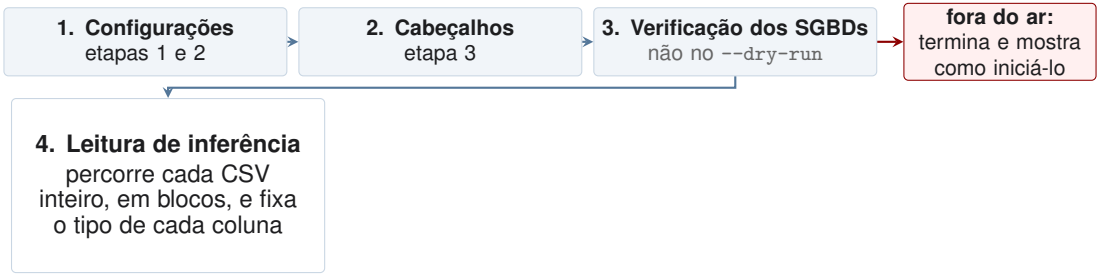
A importação valida as configurações e os cabeçalhos e lê os dados **três vezes**:



Passos do fluxo (Figura 11 do relatório); os passos 6 a 9 se repetem para cada SGBD destino.

Fluxo de execução

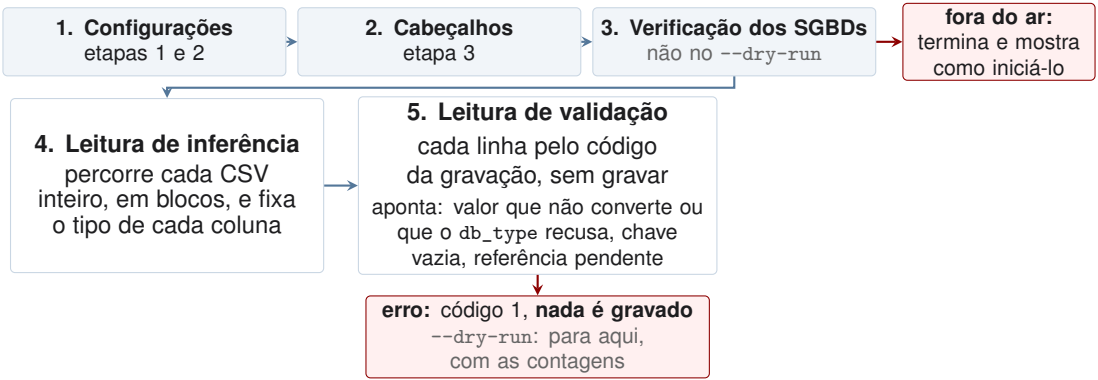
A importação valida as configurações e os cabeçalhos e lê os dados **três vezes**:



Passos do fluxo (Figura 11 do relatório); os passos 6 a 9 se repetem para cada SGBD destino.

Fluxo de execução

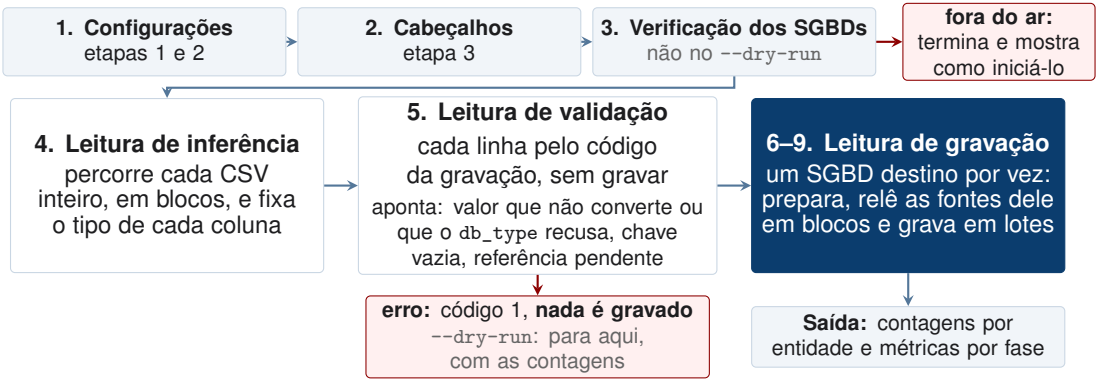
A importação valida as configurações e os cabeçalhos e lê os dados **três vezes**:



Passos do fluxo (Figura 11 do relatório); os passos 6 a 9 se repetem para cada SGBD destino.

Fluxo de execução

A importação valida as configurações e os cabeçalhos e lê os dados **três vezes**:



Passos do fluxo (Figura 11 do relatório); os passos 6 a 9 se repetem para cada SGBD destino.

Escrita em lote e leitura em blocos

Escrita em lote
como os valores são convertidos e gravados

Antes: linha a linha: cada célula convertida e cada linha gravada por um comando próprio.

Converte uma coluna inteira de uma vez e grava muitos registros por comando.

9,3× **mais rápida** (14,54 contra 135,74 s, em dez mil linhas).

Leitura em blocos
como os arquivos ocupam a memória

Antes: cada fonte inteira na memória: o pico crescia com o tamanho do arquivo.

Lê cada arquivo em blocos e grava em lotes: o pico acompanha o bloco.

Pico 12,5× **menor** (75,9 contra 950,7 MB, em cem mil linhas), com 10% a mais de tempo.

Fonte: §4.5 do relatório; ganhos medidos com uma versão anterior da ferramenta.

Escrita em lote e leitura em blocos

Escrita em lote
como os valores são convertidos e gravados

Antes: linha a linha: cada célula convertida e cada linha gravada por um comando próprio.

Converte uma coluna inteira de uma vez e grava muitos registros por comando.

9,3× **mais rápida** (14,54 contra 135,74 s, em dez mil linhas).

Leitura em blocos
como os arquivos ocupam a memória

Antes: cada fonte inteira na memória: o pico crescia com o tamanho do arquivo.

Lê cada arquivo em blocos e grava em lotes: o pico acompanha o bloco.

Pico 12,5× **menor** (75,9 contra 950,7 MB, em cem mil linhas), com 10% a mais de tempo.

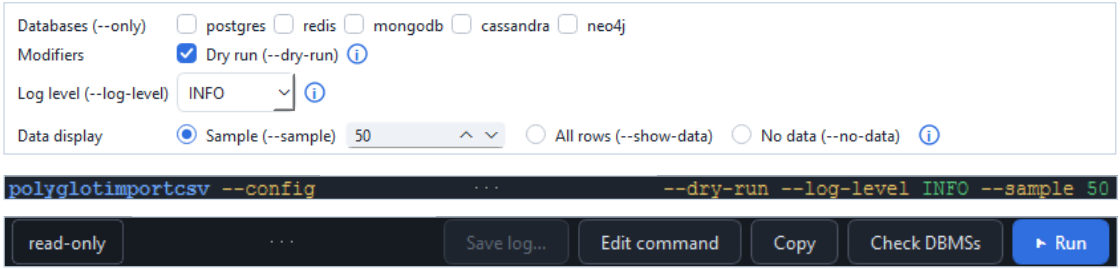
Bloco escolhido a cada execução: o que lê a maior fonte inteira, entre 16 e 256 MB, desde que o pico previsto caiba em 25% da memória livre; senão, o maior que cabe. **Lote:** 2.048 linhas.

Fonte: §4.5 do relatório; ganhos medidos com uma versão anterior da ferramenta.

Interface gráfica

O comando exibido é o comando executado.

recortes das Figuras 13, 15 e 17 do relatório



Ocioso: um controle para cada opção de uso cotidiano da CLI, com ícone de ajuda, e o comando montado, colorido e somente leitura.

Interface gráfica

O comando exibido é o comando executado.

recortes das Figuras 13, 15 e 17 do relatório

```
  [redis] user_session: 8 row(s)
  [neo4j] :PURCHASED: 8 row(s)
  [neo4j] Product: 8 row(s)
  [neo4j] User: 8 row(s)
Import metrics
```

dbms	entity	phase	rows	seconds	rows/s
(sources)	*	infer	32	0.531	60
(all)	*	validate	32	0.696	46

```
  ✓ Finished dry-run — nothing was written
```

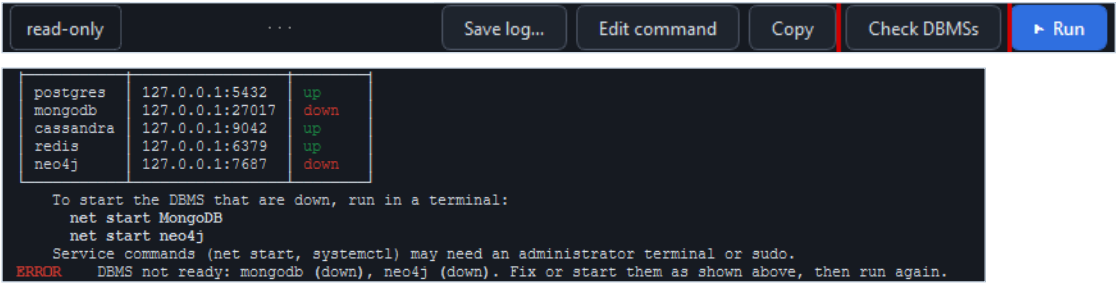
Finished — 6.45 s · · · D:\UFSC\TCC1\polyglot-import-csv\logs\polyglotimportcsv_20261110_092331.log

Simulação concluída (`--dry-run`): contagens, métricas por fase, desfecho e arquivo de log da sessão na barra de estado.

Interface gráfica

O comando exibido é o comando executado.

recortes das Figuras 13, 15 e 17 do relatório



Verificação dos SGBDs (botão *Check DBMSs*): MongoDB e Neo4j fora do ar e, abaixo da tabela de estados, os comandos que os iniciam.

Roteiro

1 Contexto e trabalhos relacionados
2 A ferramenta Polyglot Import CSV
3 Avaliação de desempenho
4 Conclusão

Metodologia da avaliação

Dados sintéticos. Gerador determinístico do cenário de e-commerce: a mesma escala N e a mesma semente geram sempre os mesmos arquivos.

Volumes. Seis, de 1.024 a 262.144 linhas; a partir de 16.384, cada um é o dobro do anterior.

Questões de pesquisa (QP)

QP1 Custo por fase e por SGBD
QP2 Multifonte *versus* combinado
QP3 Escalabilidade
QP4 Custo da validação prévia
QP5 Tamanhos do bloco e do lote

Fonte: §5.1 e §5.2 do relatório.

Metodologia da avaliação

Dados sintéticos. Gerador determinístico do cenário de e-commerce: a mesma escala N e a mesma semente geram sempre os mesmos arquivos.

Volumes. Seis, de 1.024 a 262.144 linhas; a partir de 16.384, cada um é o dobro do anterior.

Repetições. Três por célula (volume e modo), com os SGBDs limpos antes; vale a mediana.

Duas campanhas. Medir a memória registra cada alocação e deixa a importação 2,5× mais lenta, de forma desigual entre as fases. Tempos: campanha sem o registro. Picos de memória: campanha com ele.

Bloco e lote. Os que a ferramenta escolhe sozinha.

Fonte: §5.1 e §5.2 do relatório.

Questões de pesquisa (QP)

QP1 Custo por fase e por SGBD
QP2 Multifonte *versus* combinado
QP3 Escalabilidade
QP4 Custo da validação prévia
QP5 Tamanhos do bloco e do lote

Ambiente: um computador

Windows 11, 8 GB de memória

Em Docker: PostgreSQL 16, Redis 7, MongoDB 7, Neo4j 5 e Cassandra 4.1, este em um nó com 1 GB de memória Java (*heap*). Cerca de 920 MB livres.

A escrita domina o tempo (QP1)

Escrita: 79,8% do tempo

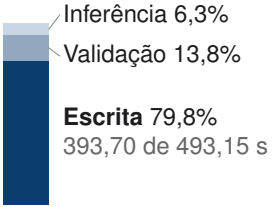


Tempo por fase de uma importação.

Fonte: Tabelas 15 e 16 do relatório; *N* = 262.144, multifonte; campanha de tempo.

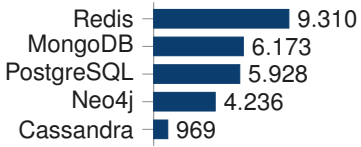
A escrita domina o tempo (QP1)

Escrita: 79,8% do tempo



Tempo por fase de uma importação.

Vazão: uma ordem de grandeza



Vazão de escrita: linhas processadas por segundo.

Fonte: Tabelas 15 e 16 do relatório; N = 262.144, multifonte; campanha de tempo.

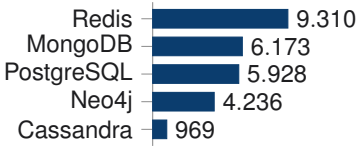
A escrita domina o tempo (QP1)

Escrita: 79,8% do tempo



Tempo por fase de uma importação.

Vazão: uma ordem de grandeza



Vazão de escrita: linhas processadas por segundo.

O Cassandra pesa mais

54,9%

da importação inteira

270,51 s dos 393,70 s da escrita: grava os eventos das quatro fontes, em um nó com 1 GB de *heap*.

Fonte: Tabelas 15 e 16 do relatório; *N* = 262.144, multifonte; campanha de tempo.

Combinado: 7% mais tempo (QP2)

Multifonte *versus* combinado

Critério	Multifonte	Combinado
Arquivos (MB)	63,26	75,92
Células vazias	0%	44%
Bloco de leitura (MB)	63	38
Inferência (s)	31,27	35,52
Validação (s)	68,18	72,08
Tempo total (s)	**493,15**	**526,17**
Pico (MB)	**253,8**	**141,9**

Tempos: campanha de tempo.
Picos: campanha de memória.

Fonte: Tabela 17 do relatório; $N = 262.144$.

Combinado: 7% mais tempo (QP2)

Multifonte *versus* combinado

Critério	Multifonte	Combinado
Arquivos (MB)	63,26	75,92
Células vazias	0%	44%
Bloco de leitura (MB)	63	38
Inferência (s)	31,27	35,52
Validação (s)	68,18	72,08
Tempo total (s)	**493,15**	**526,17**
Pico (MB)	**253,8**	**141,9**

Tempos: campanha de tempo.
Picos: campanha de memória.

Fonte: Tabela 17 do relatório; N = 262.144.

Tempo: o arquivo combinado é 20% maior, e cada SGBD relê o arquivo inteiro na gravação. No Redis, a escrita passa de 13,95 para 30,39 s.

Memória: o pico menor vem do bloco (38 contra 63 MB), que a memória livre limitou mais no combinado, e não do modo.

Combinado: 7% mais tempo (QP2)

Multifonte *versus* combinado

Critério	Multifonte	Combinado
Arquivos (MB)	63,26	75,92
Células vazias	0%	44%
Bloco de leitura (MB)	63	38
Inferência (s)	31,27	35,52
Validação (s)	68,18	72,08
Tempo total (s)	**493,15**	**526,17**
Pico (MB)	**253,8**	**141,9**

Tempos: campanha de tempo.
Picos: campanha de memória.

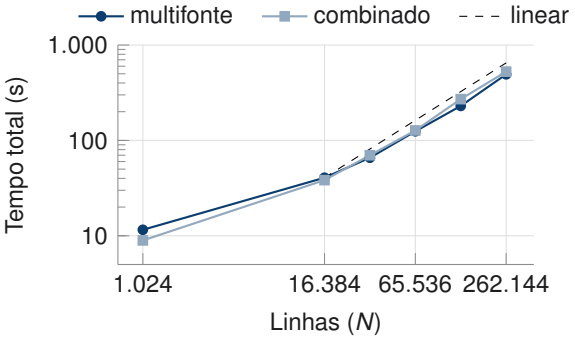
Fonte: Tabela 17 do relatório; N = 262.144.

Tempo: o arquivo combinado é 20% maior, e cada SGBD relê o arquivo inteiro na gravação. No Redis, a escrita passa de 13,95 para 30,39 s.

Memória: o pico menor vem do bloco (38 contra 63 MB), que a memória livre limitou mais no combinado, e não do modo.

Organização: no multifonte, cada arquivo traz só as colunas do seu evento, e `--source` substitui uma fonte isolada. O combinado é um arquivo só.

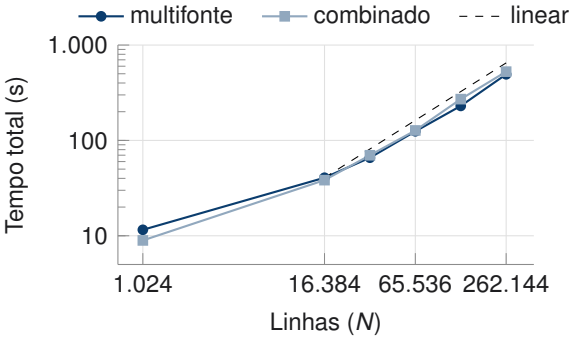
O tempo cresce linearmente com o volume (QP3)



Tempo total por volume, em escala logarítmica nos dois eixos.

Fonte: Tabela 18 e Figura 18 do relatório. Tempos: campanha de tempo. Picos: campanha de memória.

O tempo cresce linearmente com o volume (QP3)



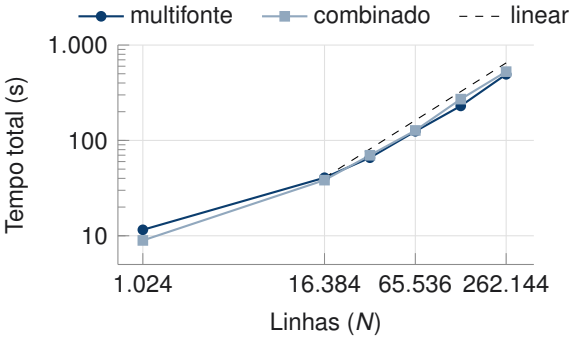
Tempo total por volume, em escala logarítmica nos dois eixos.

Fonte: Tabela 18 e Figura 18 do relatório. Tempos: campanha de tempo. Picos: campanha de memória.

Tempo por linha estável: de 1,8 a 2,5 ms a partir de 16.384 linhas.

Expoente do crescimento: 0,90 no multifonte e 0,95 no combinado (1 é o crescimento proporcional ao volume).

O tempo cresce linearmente com o volume (QP3)



Tempo total por volume, em escala logarítmica nos dois eixos.

Tempo por linha estável: de 1,8 a 2,5 ms a partir de 16.384 linhas.

Expoente do crescimento: 0,90 no multifonte e 0,95 no combinado (1 é o crescimento proporcional ao volume).

A memória acompanha o bloco, não o volume: com 256 vezes mais dados, o pico vai de 10,5 para 253,8 MB (24 vezes) no multifonte.

Quando a memória livre limita o bloco, o pico deixa de acompanhar o volume: no combinado, cai de 209,3 para 141,9 MB quando o volume dobra.

Fonte: Tabela 18 e Figura 18 do relatório. Tempos: campanha de tempo. Picos: campanha de memória.

Validação prévia e bloco de leitura (QP4, QP5)

Inferência e validação, em % do tempo total

Linhas	Multifonte	Combinado
1.024	28,9%	29,3%
16.384	19,3%	22,9%
32.768	19,1%	23,0%
65.536	17,9%	19,3%
131.072	17,9%	18,2%
262.144	20,2%	20,4%

Em troca: nenhum problema detectado chega a uma gravação parcial.

Fonte: Tabelas 19, 20 e 21 do relatório. Bloco e lote: $N = 131.072$, multifonte.

Validação prévia e bloco de leitura (QP4, QP5)

Inferência e validação, em % do tempo total

Linhas	Multifonte	Combinado
1.024	28,9%	29,3%
16.384	19,3%	22,9%
32.768	19,1%	23,0%
65.536	17,9%	19,3%
131.072	17,9%	18,2%
262.144	20,2%	20,4%

Em troca: nenhum problema detectado chega a uma gravação parcial.

Tamanho do bloco de leitura (campanha exploratória)

Bloco (MB)	Tempo (s)	Diferença	Pico (MB)
16	275,08	+10,9%	103,6
32	304,36	+22,7%	127,9
64	282,19	+13,8%	144,9
128	**248,00**	—	**195,5**
256	251,00	+1,2%	195,5

Bloco: ler a maior fonte inteira (95,5 MB) é o mais rápido; um bloco maior só ocupa memória.
Lote: de 512 a 8.192 linhas, sem efeito acima do ruído da medição.

Daí sai a **escolha automática** dos dois tamanhos.

Fonte: Tabelas 19, 20 e 21 do relatório. Bloco e lote: $N = 131.072$, multifonte.

Roteiro

1 Contexto e trabalhos relacionados
2 A ferramenta Polyglot Import CSV
3 Avaliação de desempenho
4 Conclusão

Contribuições

Objetivo específico	Evidência no trabalho
1. **Configuração** em JSON: filtros, chave primária e nome da coluna.	Arquivo de importação com `filters`, `is_key` e `schema_column`; arquivo de conexão à parte; cada um com o seu JSON Schema.

Contribuições

Objetivo específico	Evidência no trabalho
1. **Configuração** em JSON: filtros, chave primária e nome da coluna.	Arquivo de importação com `filters`, `is_key` e `schema_column`; arquivo de conexão à parte; cada um com o seu JSON Schema.
2. **Validação prévia**: um erro na configuração ou nos dados impede qualquer gravação.	Três etapas de validação antes de qualquer gravação: os arquivos, a consistência entre eles e os próprios dados; verificação dos SGBDs.

Contribuições

Objetivo específico	Evidência no trabalho
1. **Configuração** em JSON: filtros, chave primária e nome da coluna.	Arquivo de importação com `filters`, `is_key` e `schema_column`; arquivo de conexão à parte; cada um com o seu JSON Schema.
2. **Validação prévia**: um erro na configuração ou nos dados impede qualquer gravação.	Três etapas de validação antes de qualquer gravação: os arquivos, a consistência entre eles e os próprios dados; verificação dos SGBDs.
3. **Avaliação** do desempenho com diferentes volumes e modelos de dados.	Cinco questões de pesquisa; de 1.024 a 262.144 linhas; os cinco SGBDs. O tempo cresce linearmente com o volume, e a validação prévia custa um quinto dele.

Contribuições

Objetivo específico	Evidência no trabalho
1. **Configuração** em JSON: filtros, chave primária e nome da coluna.	Arquivo de importação com `filters`, `is_key` e `schema_column`; arquivo de conexão à parte; cada um com o seu JSON Schema.
2. **Validação prévia**: um erro na configuração ou nos dados impede qualquer gravação.	Três etapas de validação antes de qualquer gravação: os arquivos, a consistência entre eles e os próprios dados; verificação dos SGBDs.
3. **Avaliação** do desempenho com diferentes volumes e modelos de dados.	Cinco questões de pesquisa; de 1.024 a 262.144 linhas; os cinco SGBDs. O tempo cresce linearmente com o volume, e a validação prévia custa um quinto dele.
4. **Acesso** por linha de comando (CLI) e por interface gráfica (GUI).	CLI com `--dry-run`, `--only` e `--check-dbms`; GUI que monta e executa o comando, mantendo-o visível.

Contribuições

Objetivo específico	Evidência no trabalho
1. **Configuração** em JSON: filtros, chave primária e nome da coluna.	Arquivo de importação com `filters`, `is_key` e `schema_column`; arquivo de conexão à parte; cada um com o seu JSON Schema.
2. **Validação prévia**: um erro na configuração ou nos dados impede qualquer gravação.	Três etapas de validação antes de qualquer gravação: os arquivos, a consistência entre eles e os próprios dados; verificação dos SGBDs.
3. **Avaliação** do desempenho com diferentes volumes e modelos de dados.	Cinco questões de pesquisa; de 1.024 a 262.144 linhas; os cinco SGBDs. O tempo cresce linearmente com o volume, e a validação prévia custa um quinto dele.
4. **Acesso** por linha de comando (CLI) e por interface gráfica (GUI).	CLI com `--dry-run`, `--only` e `--check-dbms`; GUI que monta e executa o comando, mantendo-o visível.

Uma ferramenta importa o mesmo CSV para cinco SGBDs de paradigmas diferentes.

Limitações e trabalhos futuros

Limitações

- **Ambiente:** um computador, com os cinco SGBDs em contêineres; pesa sobretudo na escrita do Cassandra.
- **Sem atomicidade entre SGBDs:** uma falha durante a gravação deixa uma carga parcial, que a ferramenta não desfaz.
- **Escolha do bloco:** conservadora demais no modo combinado.
- **Testes:** a maioria é unitária.

Fonte: Cap. 6 do relatório.

Limitações e trabalhos futuros

Limitações

- **Ambiente:** um computador, com os cinco SGBDs em contêineres; pesa sobretudo na escrita do Cassandra.
- **Sem atomicidade entre SGBDs:** uma falha durante a gravação deixa uma carga parcial, que a ferramenta não desfaz.
- **Escolha do bloco:** conservadora demais no modo combinado.
- **Testes:** a maioria é unitária.

Trabalhos futuros

- **Edição da configuração na GUI**, com validação durante a digitação.
- **Avaliação:** milhões de linhas, SGBDs dedicados e ajustados, e o modelo de pico ajustado ao modo combinado.
- **Outras entradas:** TSV e planilhas Excel; coerção de tipos configurável.
- **Novos destinos** no mesmo núcleo de importação, como um motor de busca (Elasticsearch).

Fonte: Cap. 6 do relatório.



Universidade Federal de Santa Catarina · Centro Tecnológico
Departamento de Informática e Estatística · Ciência da Computação

Obrigado! Perguntas?

Lucas Bueno Cesario

Exemplo de execução

```
python -m polyglotimportcsv \                          ← executa a ferramenta
  --config data/ecommerce/import_config.json \         ← arquivo de importação
  --dbms-config data/ecommerce/dbms_config.json \      ← arquivo de conexão
  --dry-run                                            ← valida e conta, sem gravar
```

Linhas a gravar, segundo o `--dry-run` — após os filtros, antes de eliminar duplicatas

SGBD	Destino	Linhas
PostgreSQL	`categories`	8
PostgreSQL	`products`	8
PostgreSQL	`inventory`	8
PostgreSQL	`orders`	8
MongoDB	`product_catalog`	8
MongoDB	`orders_state_SP`	2
MongoDB	`orders_state_CA`, RJ, RS, SC, SE e TO	1 cada

SGBD	Destino	Linhas
Cassandra	`user_activity_log`	32 (união das 4 fontes)
Redis	`shopping_cart`	8
Redis	`user_session`	8
Neo4j	nó `User`	8
Neo4j	nó `Product`	8
Neo4j	aresta `PURCHASED`	8 (uma por linha da fonte `purchase`)

Fonte: Tabela 12 do relatório.

Falha no meio da importação

- **Antes de gravar**, as três etapas de validação conferem configuração e dados, e todos os SGBDs destino precisam responder. Qualquer falha: nada é gravado.
- **Um SGBD por vez**, em lotes: PostgreSQL, MongoDB, Cassandra, Redis, Neo4j.
- **Sem atomicidade entre SGBDs** (em aberto, carta B3): não há transação comum nem desfazer (*rollback*).
- **Falha durante a gravação**, como a queda de um SGBD: os anteriores ficam gravados, o atual fica parcial, os seguintes ficam vazios; a saída lista os já gravados.
- **Reexecução** com `--only`: com chave, PostgreSQL, Cassandra, Redis e Neo4j não duplicam. No MongoDB, sem `_id` mapeado, duplica documentos; com `_id`, para na chave repetida.

Resolve × em aberto

O artigo trata de **evolução de esquemas**; este trabalho, de **importação**.
Três questões do artigo, levadas para a importação (ARAUJO; SILVA; MELLO, 2025):

Questão do artigo	Na importação, com a Polyglot Import CSV	Situação
Gerar instruções adequadas a cada SGBD	Um só arquivo de importação gera a escrita de cada SGBD: SQL, CQL (Cassandra), Cypher (Neo4j), comandos do Redis e documentos do MongoDB.	**resolve**
Estimar o efeito das modificações antes que sejam executadas	Validação prévia e `--dry-run`: contagens por SGBD e por entidade, sem gravar.	**resolve**
Garantir atomicidade (tudo ou nada)	A validação prévia barra os erros de configuração e de dados, mas a gravação vai de um SGBD por vez, sem transação comum: uma falha durante ela, como a queda de um SGBD, deixa uma carga parcial.	**em aberto**

Coerção de tipos

O CSV é lido como **texto**. A **leitura de inferência** percorre o arquivo inteiro e classifica cada coluna, nesta ordem:

1. Booleano
todos os valores não vazios são `true` ou `false`

→

2. Número
todos são inteiros ou reais

→

3. Data e hora
mais de 85% dos não vazios são datas

→

4. Texto
nenhum dos casos anteriores

- Célula vazia vira nulo em número, booleano e data; em texto, fica vazia.
- Valor só com dígitos nunca conta como data: uma coluna de IDs não vira data.
- A **leitura de validação** aponta, com arquivo, linha, coluna e valor, o valor que não converte para o tipo da coluna e o que o tipo declarado (`db_type`) recusaria. Nada é gravado.
- No PostgreSQL, chave repetida não duplica; chave vazia é apontada na validação.

Política de coerção configurável: trabalho futuro (Cap. 6 do relatório).

Por que configurar à mão?

Mapeamento declarativo
o que a ferramenta faz hoje

- Quem projeta decide que dado vai para cada SGBD, com chaves, partições e arestas.
- O arquivo pode ser revisto, e o `--dry-run` o valida sem gravar.
- A mesma configuração reproduz a mesma carga.
- Parte já é automática: sem `columns`, entram todas as colunas, com tipos inferidos.

Mapeamento inferido
por um modelo de linguagem (LLM)

- Poupa a escrita inicial do arquivo.
- O CSV não diz como a aplicação consulta os dados. É isso que decide o SGBD de cada dado.
- A proposta ainda precisa de revisão.

Complementares: um LLM poderia propor o arquivo de importação, e o `--dry-run` o conferiria sem gravar. Caminho possível, fora do escopo deste trabalho.

Importadores nativos: o que cada um exige

SGBD	Importador	O que exige
PostgreSQL	`COPY`	colunas na ordem do CSV; arquivo na máquina do servidor; permissão de superusuário
Redis	`redis-cli --pipe`	arquivo prévio com os comandos (`SET chave valor`); resumo só com os totais de erros e respostas
MongoDB	`mongoimport`	banco, coleção e formato por parâmetros; uma coleção por comando
Cassandra	`cqlsh`, `sstableloader`	tipos e colunas iguais aos da tabela; o `sstableloader` exige SSTables (formato próprio)
Neo4j	`neo4j-admin import`	CSVs separados para nós e para relacionamentos; banco novo, vazio
Polyglot Import CSV		**o mesmo CSV nos cinco SGBDs: um arquivo de importação, validação prévia e `--dry-run`**

Comparação qualitativa: o Cap. 5 não mede esses importadores. Fonte: §3.1 do relatório.

Cassandra e o ambiente

Escrita por SGBD — linhas processadas

SGBD	Linhas	Tempo (s)	Linhas/s
Cassandra	262.144	**270,51**	**969**
Neo4j	234.027	55,25	4.236
PostgreSQL	193.096	32,57	5.928
MongoDB	132.230	21,42	6.173
Redis	129.914	13,95	9.310

O Cassandra processa mais linhas e mais devagar: **270,51 s dos 393,70 s** da escrita, 68,7%.

Fonte: Tabela 16 do relatório; N = 262.144, multifonte; campanha de tempo.

Ambiente (§5.2)

Um computador, 8 GB de memória, os cinco SGBDs em contêineres Docker. Cassandra: um nó, *heap* de 1 GB numa máquina virtual Docker de 3,88 GiB.

Leitura do relatório (§5.4.1)

As 969 linhas/s refletem o *heap* pequeno: restrição do ambiente, não do SGBD.

Escala: trabalho futuro

SGBDs dedicados e ajustados; escrita do Cassandra além da concorrência atual.

A escolha automática do bloco

Bloco escolhido na campanha final, em MB

Linhas	Multifonte	Combinado
1.024	16	16
16.384	16	35
32.768	24	71
65.536	48	141
131.072	97	116*
262.144	63*	38*

* Limitado pela memória livre; nos demais, o bloco lê a maior fonte inteira (16 MB é o mínimo).

Fonte: §4.5.2, §5.3.5 e Tabela 22 do relatório.

A regra

Bloco que lê a maior fonte inteira, entre 16 e 256 MB. Pico previsto: uma base que cresce com os arquivos, mais cerca de 1,08 MB por MB de bloco; deve caber em 25% da memória livre.

O modelo conferido

orçamento: 230,5 MB (25% de 922 MB livres)

Multifonte, 262.144 linhas: pico de 253,8 MB, 10% acima.

Combinado: 209,3 MB em 131.072 linhas e 141,9 MB em 262.144, este 38% abaixo: o modelo superestima o pico.

Ameaças à validade

De construção
mede o que se quer?

- Tempo e memória de campanhas separadas, com a mesma carga.
- Só a mediana é guardada; na campanha exploratória, a mesma célula variou de 3,5% a 24%.
- Campanha de tempo pausada e retomada com o Docker reiniciado.

Interna
o efeito vem do fator?

- Gerador determinístico e SGBDs limpos antes de cada execução.
- Volume moderado: até 262.144 linhas.
- Sem caches quentes nem execuções concorrentes.

Externa
vale fora deste ambiente?

- Um computador, cinco SGBDs em contêineres com 8 GB: não é produção.
- Cassandra limitado pelo *heap* de 1 GB.
- O bloco depende da memória livre do computador.
- Dados sintéticos de um só domínio (§5.1.1).

Fonte: §5.4.1 do relatório.

Referências

ARAUJO, L. N.; SILVA, H. A. B. d.; MELLO, R. d. S. Uma revisão sistemática simplificada e análise de estudos sobre evolução de esquemas em aplicações de persistência poliglota. In: *Anais da Escola Regional de Banco de Dados (ERBD)*. [S.l.]: Sociedade Brasileira de Computação, 2025. DOI: 10.5753/erbd.2025.7322.

PEREIRA, F. d. O.; GUERRA, E.; ROSA, R. R. A framework model for supporting transparent polyglot persistence with a unified API and extensible for different database types. In: *Proceedings of the 27th International Conference on Enterprise Information Systems (ICEIS)*. [S.l.]: SciTePress, 2025. p. 109–120. DOI: 10.5220/0013369300003929.

SACHDEVA, S.; VASAVA, J. Plugging and playing with variety of data using multi-model database and polyglot persistence. In: *2024 Third International Conference on Electrical, Electronics, Information and Communication Technologies (ICEEICT)*. [S.l.]: IEEE, 2024. p. 1–8. DOI: 10.1109/ICEEICT61591.2024.10718481.